

From CUDA Cores to Production

GPU Systems for Deep Learning

Department of Computer Science and Engineering
Indian Institute of Technology Kharagpur



What We'll Cover

- **GPU Architecture & CUDA Execution Model**
 - Threads, warps, SMs, and the memory hierarchy
 - How data moves between CPU and GPU
- **cuDNN, cuBLAS & Framework Execution**
 - Where PyTorch dispatches your matmuls and convolutions
 - Tensor Cores and mixed precision
- **Profiling & Performance Engineering**
 - Finding the real bottleneck before fixing it
 - Nsight, the PyTorch Profiler, and the roofline model
- **PyTorch Optimization Patterns**
 - torch.compile, DDP, and memory management
 - Common performance pitfalls
- **Serving: FastAPI & GPU Inference**
 - Concurrency vs. parallelism
 - Dynamic batching and serving architectures
- **MLOps: Tracking, Registry, CI/CD/CT, Monitoring**
 - Closing the loop from production back into training

Why does deep learning even need a different kind of chip?

Why GPUs for Deep Learning?

Deep learning is dominated by dense linear algebra: matrix multiplies and convolutions with millions of independent operations.

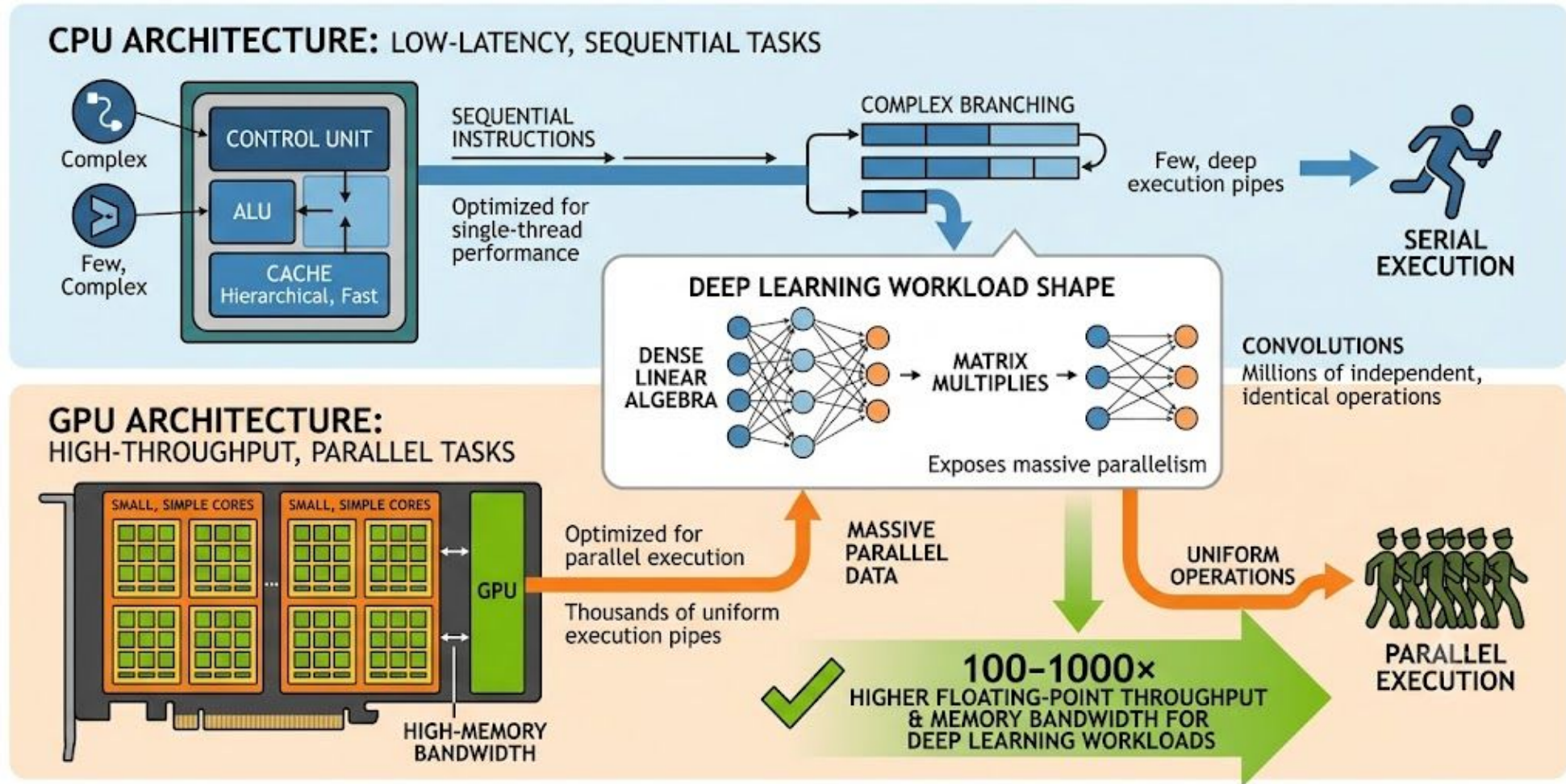
- CPUs target low-latency, complex instruction streams; GPUs target high-throughput, massively parallel workloads.

Modern GPUs can deliver **100-1000×** higher throughput and memory bandwidth than CPUs for these workloads.

*The catch: GPUs require massive, **regular** parallelism, which neural networks naturally provide.*

The Shape of a Deep Learning Workload

WHY DEEP LEARNING DOMINATES ON GPUS



CPU vs. GPU: Two Different Bets

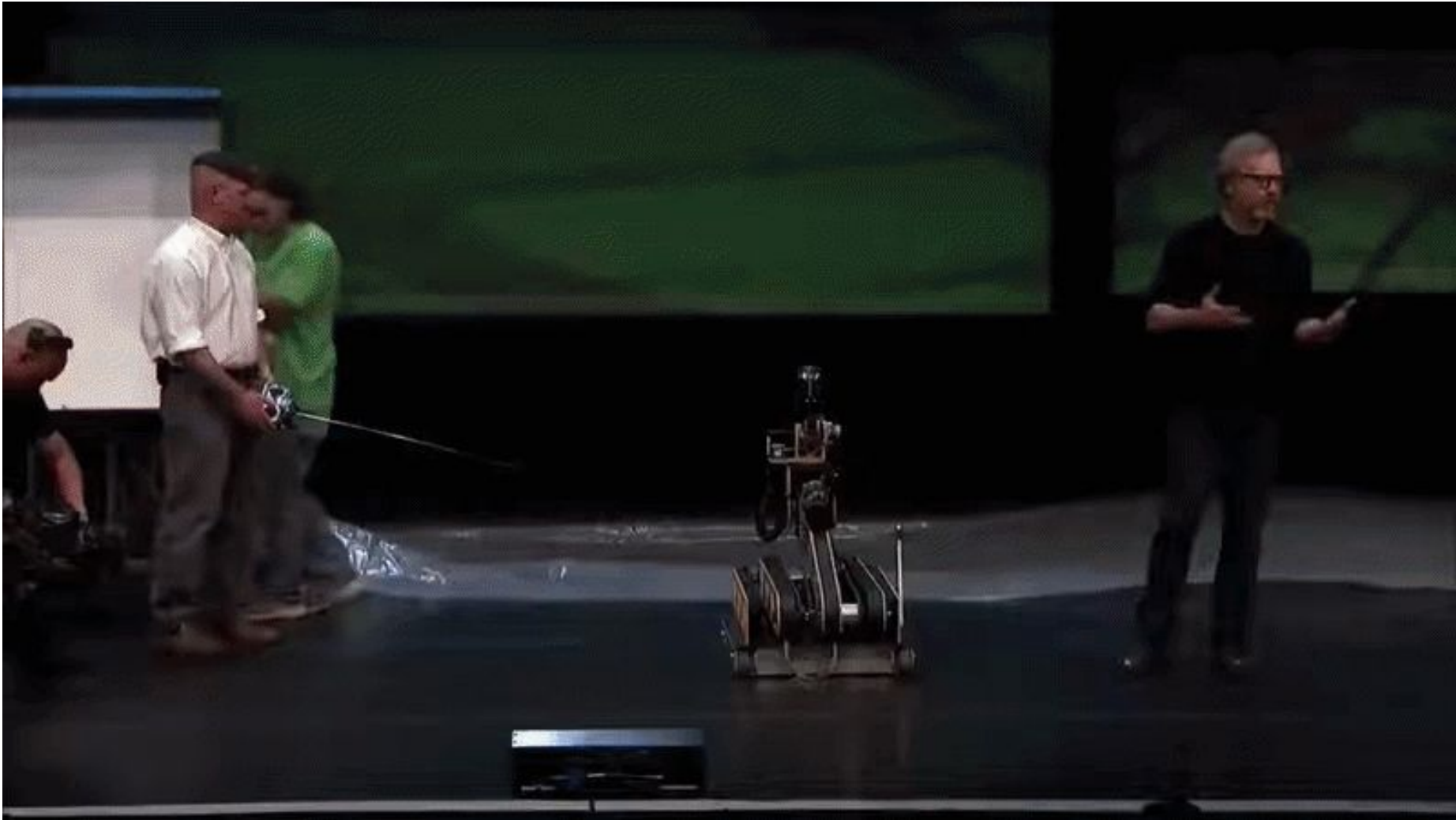
CPU: Latency-Optimized

- Few cores (as few as single digits to dozens), each very powerful
- Large caches, deep out-of-order pipelines
- Branch prediction, speculative execution
- Optimized for one thread finishing fast
- Great for control-heavy, sequential logic

GPU: Throughput-Optimized

- Thousands of simple cores (CUDA cores)
- Small caches, in-order, massively multithreaded
- Latency hidden by switching between warps
- Optimized for total work finished per second
- Great for data-parallel, regular math

CPU vs. GPU: Two Different Bets



<https://www.youtube.com/watch?v=gJ84mx3HbY8>

SIMT: Single Instruction, Multiple Threads

GPUs execute in a SIMT model: groups of threads execute the **same instruction** on **different data**, in lockstep, at each clock cycle.

- This is a middle ground between SIMD (one instruction stream, no per-thread control flow) and MIMD (fully independent threads): each thread has its own registers and program counter, but they execute together in hardware groups.

The hardware grouping unit is the **warp** (32 threads on NVIDIA GPUs): the true unit of scheduling, not the individual thread.

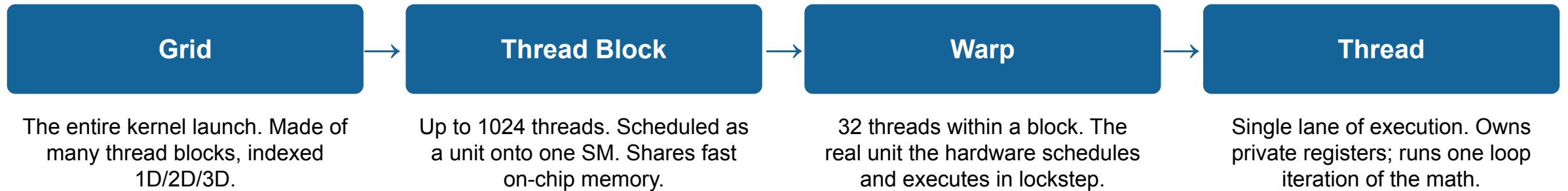
- Divergent branches (if/else that threads disagree on) within a warp force serialized execution of both paths — a key performance pitfall we'll return to.

Key terms

- SIMT = SIMD + per-thread state
- Warp = 32 threads, one scheduling unit
- Warp divergence = both branch paths run
- Occupancy = active warps / max warps

The CUDA Execution Hierarchy

A kernel launch decomposes into a grid of blocks of threads — this hierarchy maps directly onto the hardware.

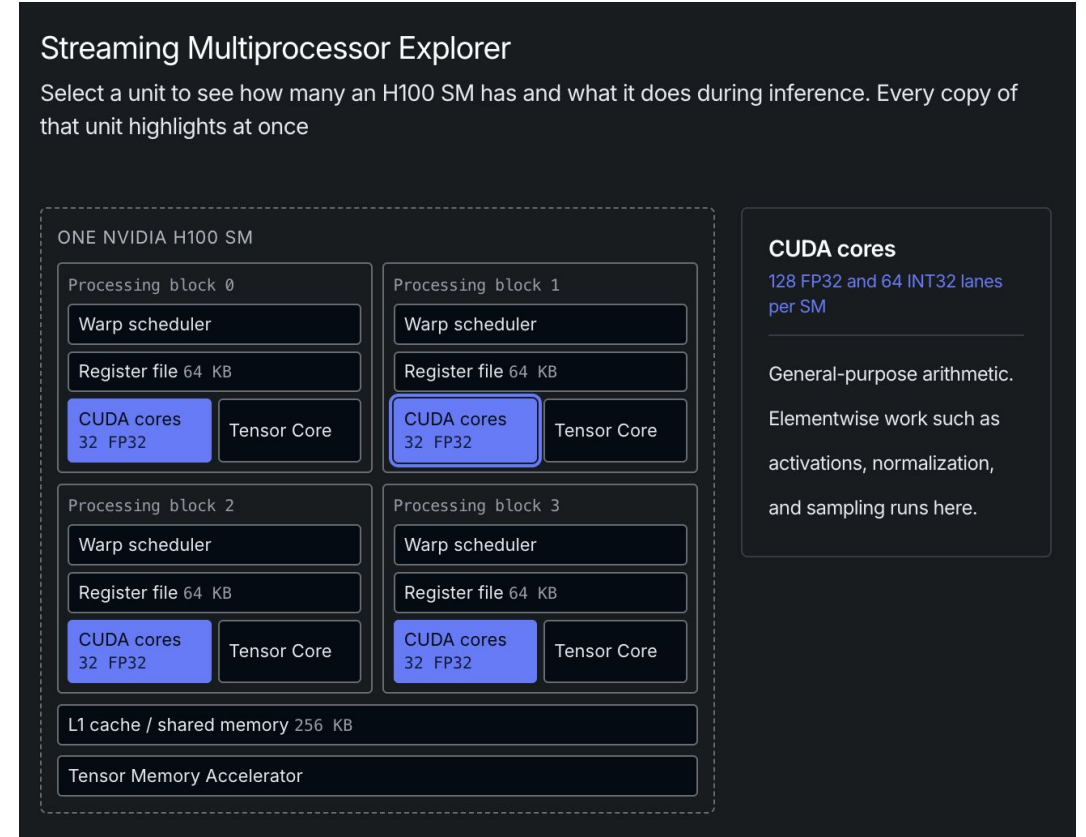


Warps: The Real Unit of Execution

- A warp is a group of 32 consecutive threads within a block that the SM's warp scheduler issues instructions to together, every cycle.
- When a warp stalls (e.g., waiting on a memory load), the scheduler instantly switches to another resident warp — this is how GPUs hide latency instead of avoiding it.
- This is why occupancy matters: an SM needs enough resident warps to always have one ready to run while others wait on memory.
- Block size is almost always chosen as a multiple of 32 — a block of 100 threads wastes 28 lanes in its last warp.

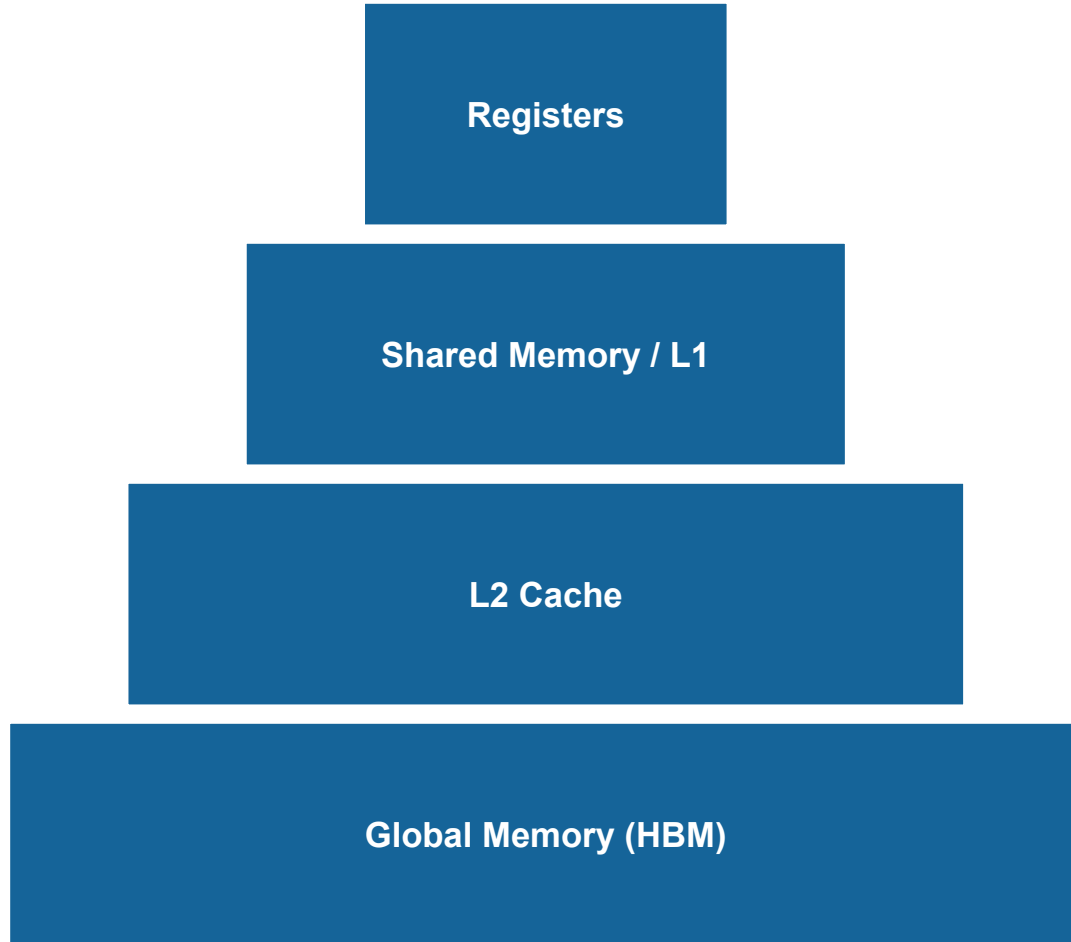
Streaming Multiprocessors (SMs)

- The SM is the GPU's core compute engine — a modern data-center GPU has 100+ SMs, each capable of holding resident warps from multiple blocks simultaneously.
- Each SM contains CUDA cores (FP32/INT32 units), Tensor Cores (matrix-multiply-accumulate units), a warp scheduler, register file, and shared memory / L1 cache.
- A thread block is assigned to exactly one SM for its entire lifetime — it cannot migrate — but an SM can host several blocks at once if resources allow.



Source: [handbook.modular.com](https://handbook.modular.com/kernel-optimization/gpu-architecture-fundamentals) — kernel-optimization / gpu-architecture-fundamentals

The GPU Memory Hierarchy



Registers: Per-thread, fastest. ~1 cycle latency. Thousands per SM, partitioned across resident threads.

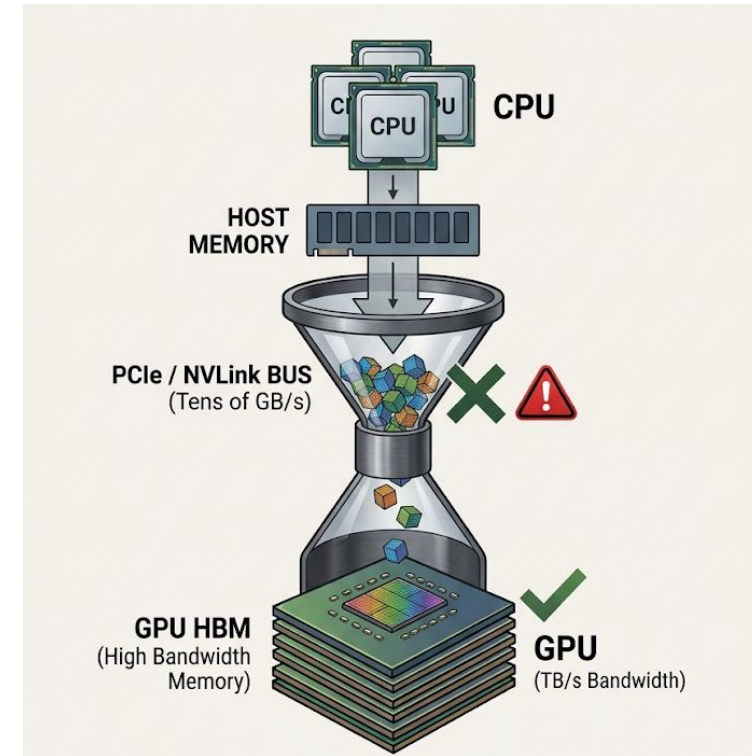
Shared Memory / L1: Per-block, on-chip. Programmer-managed scratchpad, ~20–30× faster than global memory.

L2 Cache: Shared across SMs. Chip-wide cache; catches reuse across blocks and kernels.

Global Memory (HBM): Off-chip, largest. High bandwidth (~TB/s) but hundreds of cycles of latency.

CPU ↔ GPU Data Movement

- The GPU is a separate device with its own memory; data must be explicitly copied across the PCIe (or NVLink) bus before a kernel can use it.
- PCIe bandwidth (tens of GB/s) is an order of magnitude below GPU HBM bandwidth (TB/s) — transfers are often the real bottleneck, not compute.
- Pinned (page-locked) host memory enables faster, asynchronous DMA transfers versus regular pageable memory.
- *Best practice: minimize transfers, batch them, overlap them with compute using streams, and keep data resident on the GPU across steps whenever possible.*



CUDA Streams & Asynchronous Execution

- A CUDA stream is an ordered queue of operations (kernel launches, memory copies) that execute in sequence on the GPU — but kernel launches from the host are asynchronous by default.
- Operations in different streams can overlap: data transfer for batch N+1 can run concurrently with compute for batch N.
- The default stream is implicitly synchronizing in older CUDA versions — a common hidden performance trap; explicit streams avoid this.
- Frameworks like PyTorch use a default stream per device automatically, but expose `torch.cuda.Stream()` for manual overlap of compute and data transfer.

Why this matters

- Hides PCIe transfer behind compute
- Enables overlapping data prefetch
- Basis for CUDA graphs & multi-stream serving
- Misuse causes silent serialization

What We'll Cover

- GPU Architecture & CUDA Execution Model
 - Threads, warps, SMs, and the memory hierarchy
 - How data moves between CPU and GPU
- **cuDNN, cuBLAS & Framework Execution**
 - **Where PyTorch dispatches your matmuls and convolutions**
 - **Tensor Cores and mixed precision**
- Profiling & Performance Engineering
 - Finding the real bottleneck before fixing it
 - Nsight, the PyTorch Profiler, and the roofline model
- PyTorch Optimization Patterns
 - torch.compile, DDP, and memory management
 - Common performance pitfalls
- Serving: FastAPI & GPU Inference
 - Concurrency vs. parallelism
 - Dynamic batching and serving architectures
- MLOps: Tracking, Registry, CI/CD/CT, Monitoring
 - Closing the loop from production back into training

How does torch.matmul comes to the GPU kernel?

From Kernel to Framework: Where PyTorch Fits

- When you call `torch.matmul` or `nn.Conv2d`, PyTorch dispatches to vendor libraries rather than hand-writing a CUDA kernel.
- The ATen backend selects an implementation based on shape, dtype, and device, then calls cuBLAS (linear algebra) or cuDNN (deep learning).
- These libraries contain many optimized algorithm variants and choose the fastest one using heuristics or autotuning (`cudaDeviceSynchronize=True`).
- **This explains why:**
 - First iteration can be slow → autotuning/setup
 - Changing batch size changes speed → different algorithm selection
 - Custom kernels rarely win → vendor libraries are heavily optimized

Call stack, top to bottom

- Python / `torch.nn` API
- ATen dispatcher (dtype, device, shape)
- cuDNN / cuBLAS / custom CUDA kernels
- Hardware: SMs, Tensor Cores, HBM

cuDNN: Convolutions at Scale

- cuDNN provides tuned implementations of convolution, pooling, normalization, activation, and RNN primitives, the building blocks of CNNs and beyond.
- Convolution has several distinct algorithms (implicit GEMM, Winograd, FFT-based) with different speed/memory trade-offs depending on kernel size, stride, and channel count.

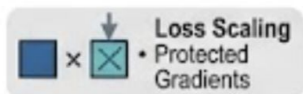
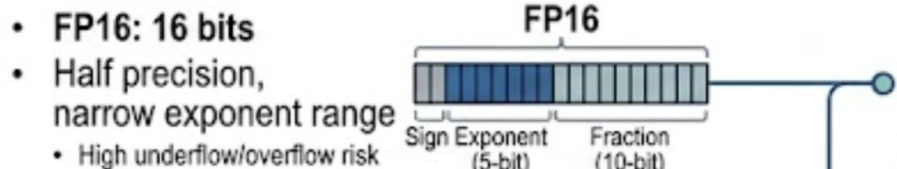
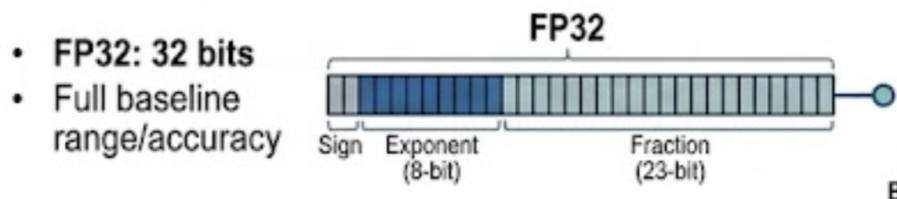
Algorithm	Best when...
Implicit GEMM	General-purpose
Winograd	Small kernels, e.g. 3×3
FFT	Large kernels

`cudnn.benchmark = True` in PyTorch triggers an autotuning pass on the first few iterations to pick the fastest algorithm for your fixed input shapes.

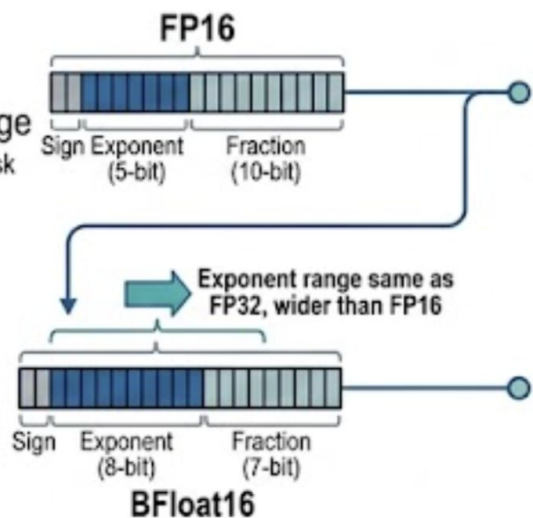
Tensor Cores & Mixed Precision

The Formats

- FP32: full precision, baseline range/accuracy
- FP16: half precision, narrow exponent range
- BF16: half precision, FP32-like exponent range



- **BF16: 16 bits**
- Half precision, FP32-like exponent range



Why It's Fast

- Tensor Cores perform fused matrix-multiply-accumulate at reduced precision natively in hardware
- Roughly 2-8× throughput vs. FP32 GEMM, plus halved memory traffic and footprint
- BF16 is generally preferred for training stability (wide exponent, less overflow risk)
- Loss scaling protects small gradients from underflowing in FP16

Automatic Mixed Precision (AMP) in PyTorch

`torch.autocast` runs compute-heavy ops (matmul/convolution) in FP16/BF16, and keeps sensitive ops (softmax/reductions) in FP32.

`GradScaler` scales the loss up before `backward()`, then unscales gradients before the optimizer step, preventing tiny FP16 gradients from underflowing.

Why AMP? 1.5–3× faster training, ~50% less activation memory, often the single highest-leverage optimization available.

- *On modern GPUs with ample BF16 range, GradScaler is often unnecessary, BF16 autocast alone is common and simpler.*

```
scaler = torch.cuda.amp.GradScaler()

for x, y in loader:
    x, y = x.cuda(), y.cuda()
    opt.zero_grad(set_to_none=True)

    with torch.autocast(
        device_type="cuda",
        dtype=torch.float16
    ):
        out = model(x)
        loss = criterion(out, y)

    scaler.scale(loss).backward()
    scaler.step(opt)
    scaler.update()
```

What We'll Cover

- GPU Architecture & CUDA Execution Model
 - Threads, warps, SMs, and the memory hierarchy
 - How data moves between CPU and GPU
- cuDNN, cuBLAS & Framework Execution
 - Where PyTorch dispatches your matmuls and convolutions
 - Tensor Cores and mixed precision
- **Profiling & Performance Engineering**
 - **Finding the real bottleneck before fixing it**
 - **Nsight, the PyTorch Profiler, and the roofline model**
- PyTorch Optimization Patterns
 - torch.compile, DDP, and memory management
 - Common performance pitfalls
- Serving: FastAPI & GPU Inference
 - Concurrency vs. parallelism
 - Dynamic batching and serving architectures
- MLOps: Tracking, Registry, CI/CD/CT, Monitoring
 - Closing the loop from production back into training

Is your intuition about the bottleneck actually right?

Why Profile Before You Optimize

- Intuition about performance is frequently wrong, engineers routinely optimize compute when the real bottleneck is memory bandwidth, data loading, or host-device transfer.
- *Profiling turns 'I think it's slow because...' into 'the trace shows 60% of wall-clock time is in this specific kernel/op.'*
- **The workflow is always: measure → identify the bottleneck class (compute, memory, data-loading, communication) → apply the matching fix → re-measure.**

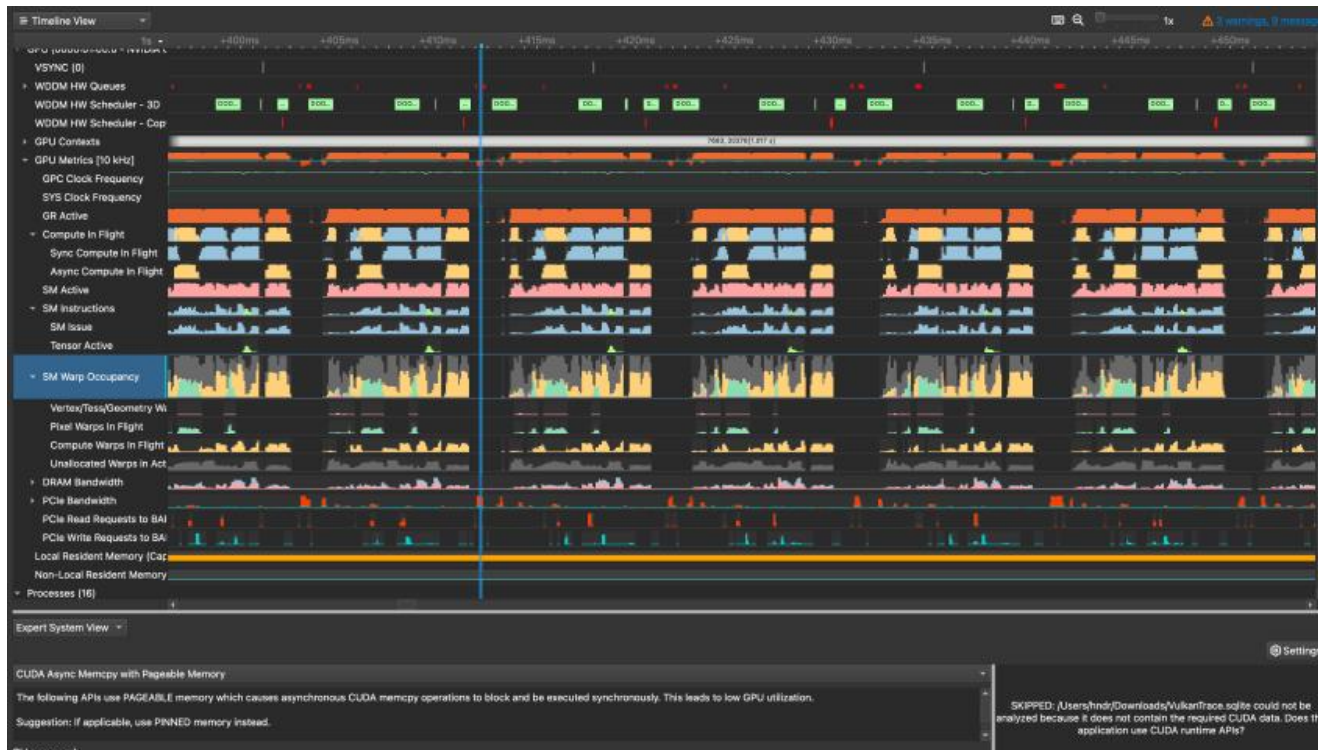
NVIDIA Nsight Systems & Nsight Compute

Nsight Systems, the timeline view

- System-wide timeline: CPU, GPU, memory copies, streams
- Shows gaps: GPU idle time waiting on the CPU or I/O
- Reveals data-loading stalls and launch overhead
- Best first tool: find which phase is slow

Nsight Compute, the kernel microscope

- Deep per-kernel analysis: occupancy, memory throughput
- Identifies register / shared-memory limiters
- Best second tool: optimize the kernel you've found



Source: <https://developer.nvidia.com/nsight-systems>
<https://developer.nvidia.com/nsight-compute>

The PyTorch Profiler in Practice

- `torch.profiler` wraps a training/inference step and records CPU ops, CUDA kernels, memory allocations, and (optionally) stack traces.
- `profiler.key_averages().table(...)` gives a quick sorted breakdown of time spent per operator — usually the fastest way to find ‘what’s slow’ without leaving Python.

```
from torch.profiler import (
    profile, ProfilerActivity
)

with profile(
    activities=[
        ProfilerActivity.CPU,
        ProfilerActivity.CUDA],
    record_shapes=True,
) as prof:
    for _ in range(5):
        train_step(model, batch)
        prof.step()

print(prof.key_averages().table(
    sort_by="cuda_time_total",
    row_limit=10))
```

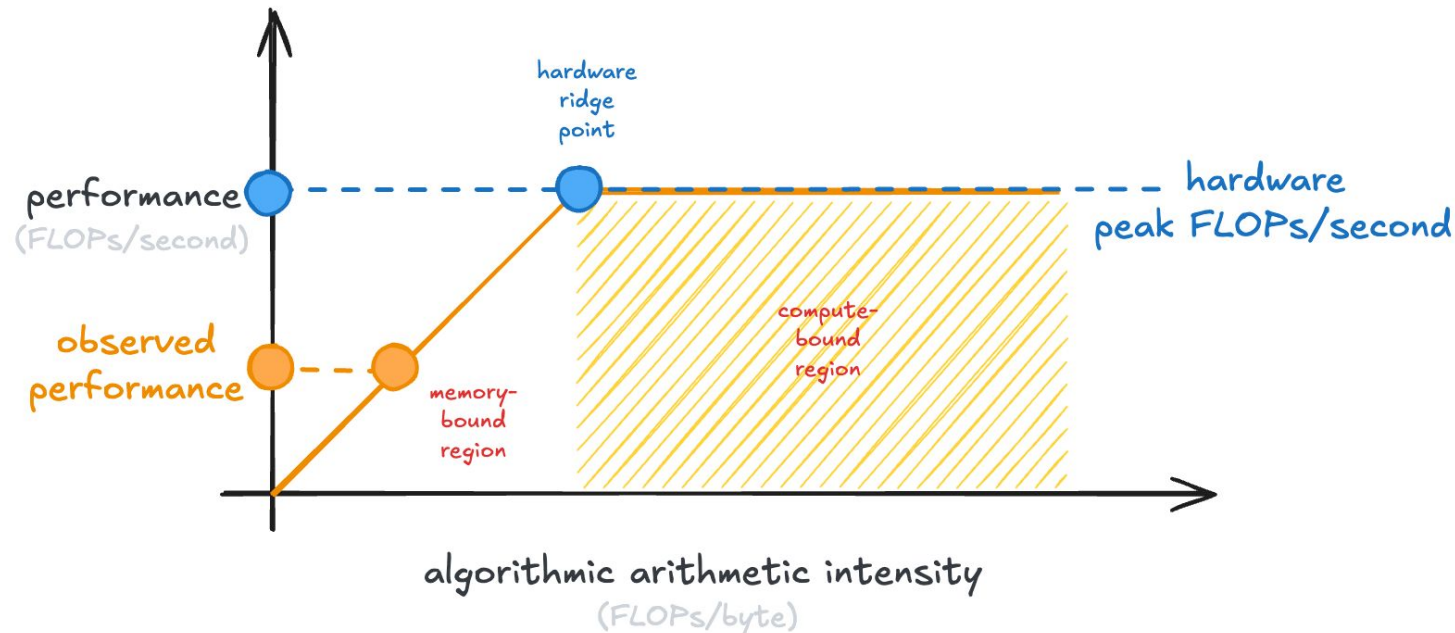
Compute-Bound vs. Memory-Bound Kernels

Compute-Bound

- GPU spends most time doing arithmetic
- High arithmetic intensity (FLOPs per byte moved)
- Example: large dense GEMMs, big convolutions
- Fix: use Tensor Cores, better algorithms

Memory-Bound

- GPU spends most time waiting for the data
- Low arithmetic intensity (few FLOPs per byte)
- Example: elementwise ops, normalization, small batches
- Fix: fusion, better access patterns, higher occupancy



Common Bottlenecks & Their Fixes

Symptom	Likely Cause	Fix
GPU utilization low, CPU busy	Data loading / preprocessing bound	More DataLoader workers, pin_memory, prefetch
High memory traffic, low FLOPs	Memory-bound kernel	Fusion, better access patterns, mixed precision
First iteration much slower	cuDNN/cuBLAS autotuning, lazy init	Warm-up iterations before timing

Case in Point: Cutting Training Time in Half

- Baseline: ResNet-50 training step at 340ms, GPU utilization reported at 55% by nvidia-smi.
- Nsight Systems trace showed large gaps between kernels, the GPU was frequently idle waiting on the DataLoader.

Fix #1: increased num_workers and enabled pin_memory + non_blocking transfers → GPU utilization rose to ~85%, step time dropped to ~210ms.

Fix #2: **enabled AMP (BF16 autocast) → step time dropped further to ~160ms, with no change in final accuracy.**

The DataLoader Bottleneck

- If the CPU can't prepare batches as fast as the GPU consumes them, the GPU sits idle — this is one of the most common real-world bottlenecks.

`num_workers > 0` spawns subprocesses to prepare batches in parallel with the current training step.

`pin_memory=True` allocates batches in page-locked host memory, enabling faster asynchronous transfer.

`non_blocking=True` on `.to('cuda')` lets the transfer overlap with other CPU work instead of blocking, when paired with pinned memory.

Checklist

- `num_workers` = a few per GPU core budget
- `pin_memory=True`
- `non_blocking=True` on device transfer
- `persistent_workers=True` to avoid respawn cost

What We'll Cover

- GPU Architecture & CUDA Execution Model
 - Threads, warps, SMs, and the memory hierarchy
 - How data moves between CPU and GPU
- cuDNN, cuBLAS & Framework Execution
 - Where PyTorch dispatches your matmuls and convolutions
 - Tensor Cores and mixed precision
- Profiling & Performance Engineering
 - Finding the real bottleneck before fixing it
 - Nsight, the PyTorch Profiler, and the roofline model
- **PyTorch Optimization Patterns**
 - **torch.compile, DDP, and memory management**
 - **Common performance pitfalls**
- Serving: FastAPI & GPU Inference
 - Concurrency vs. parallelism
 - Dynamic batching and serving architectures
- MLOps: Tracking, Registry, CI/CD/CT, Monitoring
 - Closing the loop from production back into training

You've found the bottleneck. Which lever do you actually pull?

torch.compile & Graph Capture

- torch.compile traces the model's operations into a computation graph (via TorchDynamo) and generates fused, optimized kernels (via TorchInductor).
- This automates much of the manual kernel fusion from the profiling section, turning many small elementwise ops into fewer, larger fused kernels.
- The first call triggers compilation overhead (similar to cuDNN autotuning), subsequent calls with the same input shapes reuse the compiled graph.
- **Dynamic/varying shapes can trigger recompilation.**

```
model = MyModel().cuda()
model = torch.compile(
    model,
    mode="reduce-overhead"
)

# first call: compiles (slow)
out = model(batch)

# subsequent calls:
# fast, fused kernels
for batch in loader:
    out = model(batch)
```

Gradient Accumulation & Activation Checkpointing

Gradient Accumulation

- Simulates a larger batch size on limited memory
- Runs several small forward pass and backward pass
- Accumulates gradients before a single optimizer step
- Trades wall-clock time for a larger effective batch

Activation Checkpointing

- Discards intermediate activations during forward
- Recomputes them during backward when needed
- Cuts activation memory at the cost of extra compute
- Enables training much deeper/larger models per GPU

DataParallel vs. DistributedDataParallel

DataParallel (legacy)

- Single process, multiple GPUs
- Replicates model each forward pass, high overhead
- Gathers outputs/gradients on one GPU (bottleneck)

DistributedDataParallel (DDP)

- One process per GPU, each with its own model replica
- Overlaps gradient all-reduce with the backward pass
- Scales near-linearly across many GPUs/nodes
- **The standard for real multi-GPU training**

https://docs.pytorch.org/tutorials/beginner/ddp_series_multigpu.html

The CUDA Caching Allocator

- PyTorch doesn't call `cudaMalloc/cudaFree` for every tensor, it maintains its own caching allocator that reuses freed GPU memory blocks.
- This avoids the high latency of real CUDA memory allocation calls, but can lead to fragmentation: `nvidia-smi` may show high 'reserved' memory even when little is 'allocated.'
- `torch.cuda.memory_summary()` distinguishes actual usage from the allocator's cache.
- Symptom of fragmentation: an out-of-memory error even though total reserved memory looks like it should be sufficient, often fixed by reducing `max_split_size_mb` or restructuring allocation patterns.

Common PyTorch Performance Pitfalls

Calling `.item()` or `.cpu()` inside the training loop: forces a synchronous GPU→CPU sync every iteration, stalling the pipeline.

Creating tensors on CPU then moving to GPU repeatedly: re-triggers the host-device transfer cost from Section 1.

Ignoring `cuda.benchmark` with fixed input shapes: misses free autotuned kernel selection from cuDNN.

Python-level loops over batch elements: defeats vectorization; let the GPU do the parallel work it's built for.

What We'll Cover

- GPU Architecture & CUDA Execution Model
 - Threads, warps, SMs, and the memory hierarchy
 - How data moves between CPU and GPU
- cuDNN, cuBLAS & Framework Execution
 - Where PyTorch dispatches your matmuls and convolutions
 - Tensor Cores and mixed precision
- Profiling & Performance Engineering
 - Finding the real bottleneck before fixing it
 - Nsight, the PyTorch Profiler, and the roofline model
- PyTorch Optimization Patterns
 - torch.compile, DDP, and memory management
 - Common performance pitfalls
- **Serving: FastAPI & GPU Inference**
 - **Dynamic batching and serving architectures**
- MLOps: Tracking, Registry, CI/CD/CT, Monitoring
 - Closing the loop from production back into training

Training got you a model. A stranger's request just arrived. Now what?

Training vs Serving

Training

- You control the batch, fixed size, offline
- Throughput (samples/sec) is the main metric
- Failures can be retried; no live user waiting
- Runs for hours/days on scheduled infrastructure

Serving

- Requests arrive one at a time, unpredictably
- Latency (p50/p99) matters as much as throughput
- **A slow or failed response is a live user experience**
- Must run continuously, autoscale, and stay available

FastAPI as an Inference Layer

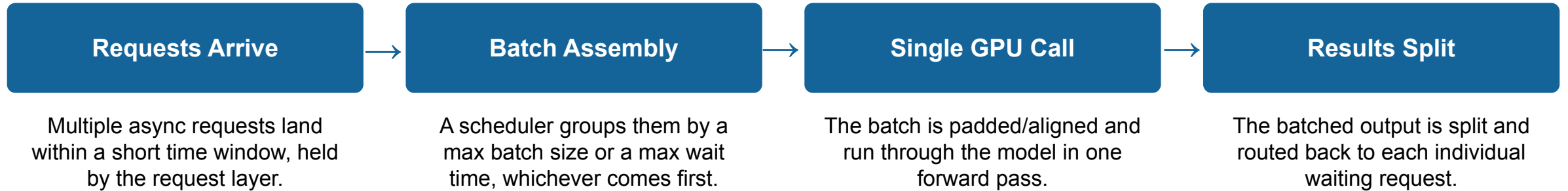
- FastAPI is an async Python web framework, a natural fit for I/O-heavy inference services that need to juggle many concurrent requests.
- Its async def endpoints let the event loop handle other requests while one is waiting on I/O (disk, network, or an async GPU call) instead of blocking a whole worker.
- *FastAPI does not make your GPU computation itself faster, it determines how well your service utilizes the GPU under concurrent load.*

Loading Models & Async Endpoints

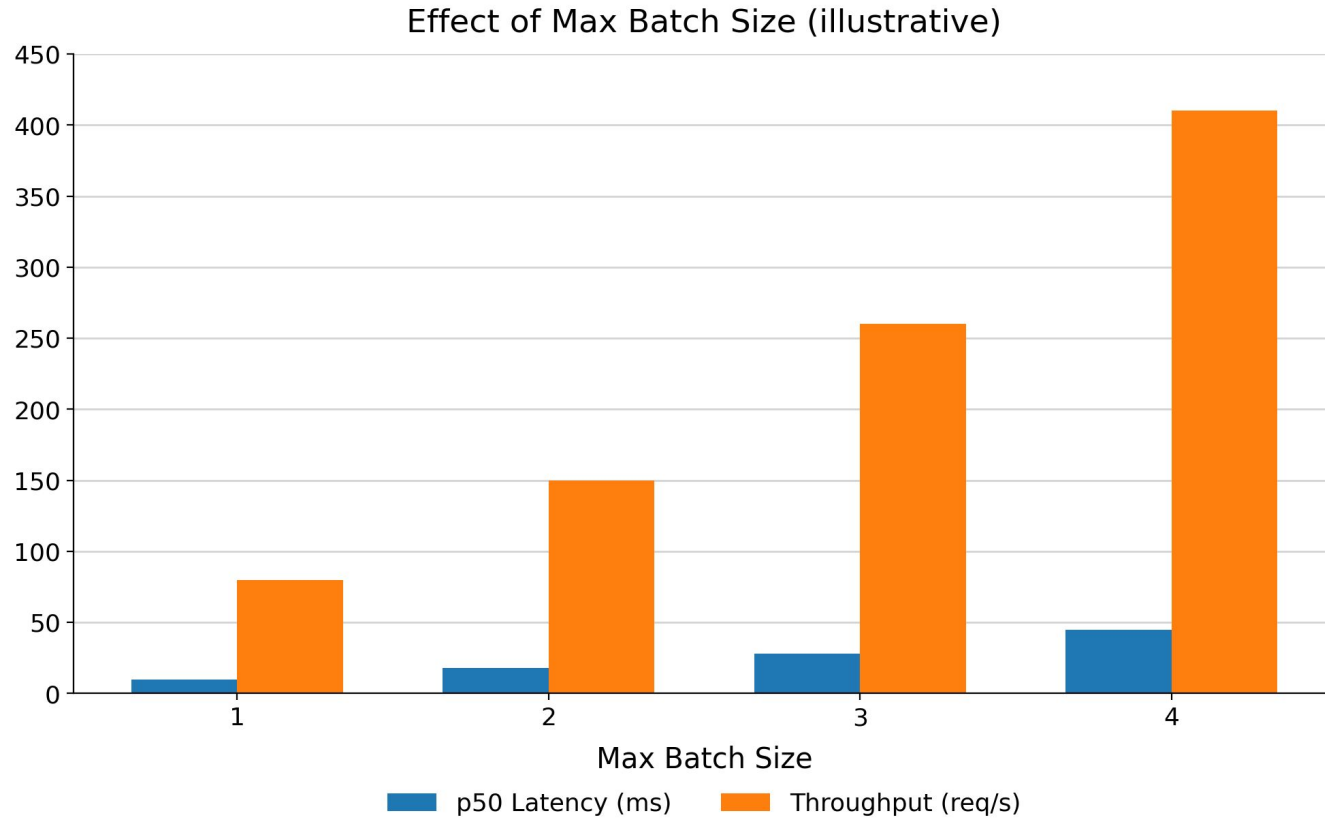
- Load the model once at application startup (a lifespan/startup event), never inside the request handler; reloading per request is a common and severe mistake.
- GPU inference calls are synchronous/blocking by nature; running them directly inside `async def` without care can block the event loop for other requests.
- Keep the model, tokenizer/preprocessor, and any warm-up (a dummy forward pass) initialized once and reused across all requests.

Dynamic Batching: How It Works

The mechanism that turns many concurrent async requests into the batched GPU calls that actually achieve parallelism.



Batching: Latency vs. Throughput



Reading the trade-off

- Larger batches → higher throughput, higher per-request latency
- Bursty traffic benefits most; steady low traffic may not

Serving Architecture Patterns

Embedded model: FastAPI process loads and runs the model directly, simplest, fine for low/moderate traffic, no cross-process batching.

Model server sidecar: FastAPI handles HTTP/validation, forwards requests to a dedicated inference server (e.g. NVIDIA Triton, TorchServe) that owns batching and GPU scheduling.

Queue-based decoupling: requests go through a message queue to a pool of GPU workers, good for bursty or long-running inference, at the cost of added latency and complexity.

Choosing a pattern

- Low traffic, simple model → embedded
- High traffic, multiple models → dedicated server
- Bursty, tolerant of ms-sec delay → queue-based
- Multiple GPUs to share → server with batching

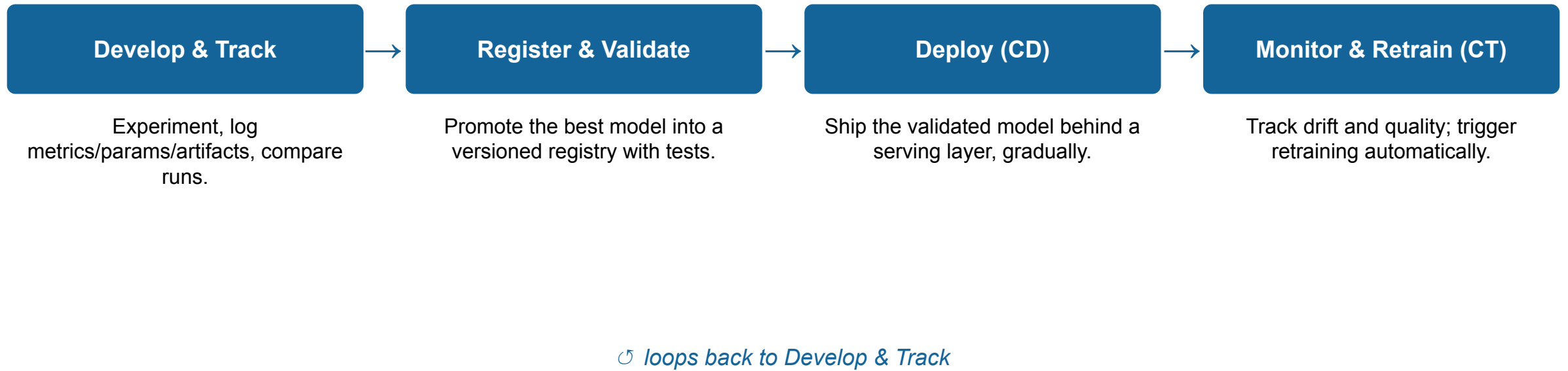
What We'll Cover

- GPU Architecture & CUDA Execution Model
 - Threads, warps, SMs, and the memory hierarchy
 - How data moves between CPU and GPU
- cuDNN, cuBLAS & Framework Execution
 - Where PyTorch dispatches your matmuls and convolutions
 - Tensor Cores and mixed precision
- Profiling & Performance Engineering
 - Finding the real bottleneck before fixing it
 - Nsight, the PyTorch Profiler, and the roofline model
- PyTorch Optimization Patterns
 - torch.compile, DDP, and memory management
 - Common performance pitfalls
- Serving: FastAPI & GPU Inference
 - Concurrency vs. parallelism
 - Dynamic batching and serving architectures
- **MLOps: Tracking, Registry, CI/CD/CT, Monitoring**
 - **Closing the loop from production back into training**

Shipped the model. Are you actually done?

The MLOps Lifecycle

A continuous loop, not a one-way pipeline: production feedback flows back into the next training run.



Experiment Tracking

- Tools like MLflow and Weights & Biases log hyperparameters, metrics, code version, and artifacts for every training run automatically.
- *This solves a concrete problem: ‘which exact configuration produced the model with 94.2% validation accuracy three weeks ago?’*
- Good tracking captures full curves (loss/accuracy over time), not just final metrics, enabling comparison across dozens of runs at a glance.
- **Tracking is a prerequisite for the model registry, you need to know why a model is good before you promote it to production.**

What to log, every run

- Hyperparameters & config
- Code / git commit hash
- Dataset version
- Metrics over time
- Model artifact + environment

Model Registry & Versioning

- A model registry is a versioned catalog of trained model artifacts, each tagged with lineage (which run/data/code produced it) and a lifecycle stage.
- The registry decouples 'a model exists' from 'a model is deployed', serving infrastructure pulls a specific registered version, never a raw training checkpoint.
- **This enables instant, auditable rollback: if a new version regresses in production, the serving layer can be pointed back at the previous registered version.**

Safe Rollouts: Canary & Shadow Deployment

Canary Release

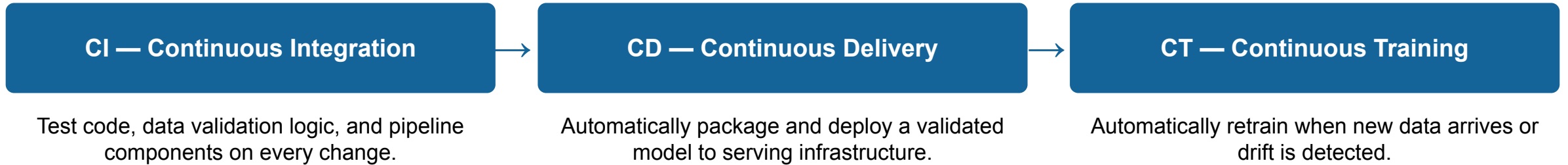
- New version serves a small % of live traffic (e.g. 5%)
- Real user impact, but limited blast radius
- Compare canary vs. baseline metrics before ramping up
- Roll back instantly to the registry's previous version if metrics regress

Shadow Deployment

- New version receives a copy of live traffic, but its output is never shown to users
- Zero user-facing risk — pure comparison against the current model
- Ideal for validating latency and prediction distribution pre-launch
- **Often the step right before a canary release**

CI/CD/CT for Machine Learning

ML pipelines add a third pillar, Continuous Training, to the familiar CI/CD pair.



Feedback loop: production monitoring signals feed back to trigger the next CT run.

Data Drift vs. Concept Drift

Data Drift

- The distribution of input features changes over time
- The relationship between inputs and labels stays the same
- Example: user demographics shift, sensor recalibration

Concept Drift

- The relationship between inputs and the correct label changes
- Input distribution may look identical
- Example: false positive in prediction

Detecting concept drift usually requires ground truth: watch for falling accuracy/quality metrics over time, since the inputs alone may look perfectly normal.

Retraining Triggers & Feedback Loops

Schedule-based: retrain on a fixed cadence (daily/weekly) regardless of detected drift, simple, predictable, sometimes wasteful.

Metric-based: retrain automatically when a monitored quality or drift metric crosses a threshold, more responsive, needs reliable ground-truth/drift signals.

Data-volume-based: retrain once enough new labeled data has accumulated to meaningfully move the model.

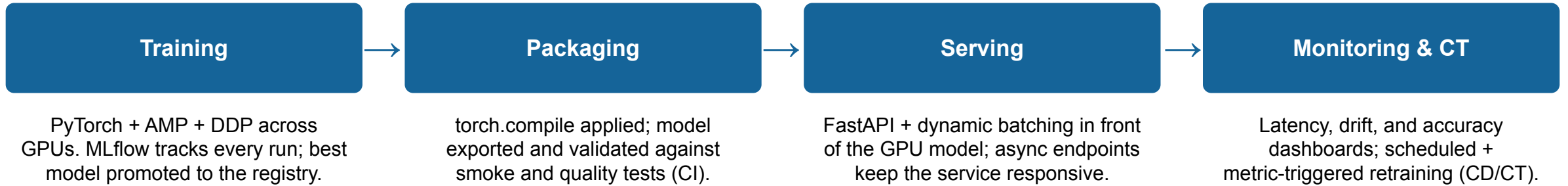
- **Whatever the trigger, the retrained model re-enters the CI/CD/CT pipeline, it doesn't skip validation just because it's automated.**

- *“A model deployed once and never revisited is a model quietly decaying in production.”*

Bringing It Together: A Production Image Classifier

End-to-End Architecture

From an uploaded image to a served prediction, and from production signal back into the next training run.



Key Takeaways

The warp, not the thread, is the real unit of GPU execution: every performance conversation traces back to keeping warps fed and SMs busy.

cuDNN/cuBLAS already solved most of the hard kernel problems: profile and configure before reaching for a custom CUDA kernel.

Profile first, optimize second, always: intuition about bottlenecks is frequently wrong; the trace is the source of truth.

MLOps closes the loop training alone never does: tracking, registry, CI/CD/CT, and monitoring keep a model correct after it ships.

Thank You!

